



SVEUČILIŠTE JOSIPA JURJA STROSSMAYERA U OSIJEKU
FAKULTET PRIMIJENJENE MATEMATIKE I INFORMATIKE

Sveučilišni prijediplomski studij Matematika i računarstvo

Baza podataka policijske stanice

SEMINARSKI RAD

Mentorica:

doc. dr. sc. Mateja Đumić

Student:

Zvonimir Šego

Osijek, 2025.

Sadržaj

1	Uvod	1
1.1	<i>MS Access</i> baza podataka policijske stanice - njeni problemi i nedostatci	1
2	MEV i relacijski model baze podataka	3
2.1	MEV model	3
2.2	Relacijski model	4
3	Kreiranje tablica	7
4	Unosi u tablice	10
5	Upiti	14
5.1	Jednostavni upiti	14
5.2	Upiti nad više tablica	16
5.3	Upiti s agregirajućim funkcijama	18
5.4	Upiti s podupitima, ugnježđenim upitima ili skupovnim operacijama .	20
6	Uvjeti i zadane vrijednosti	23
6.1	Uvjeti	23
6.2	Zadane vrijednosti	24
7	Procedure	25
8	Okidači	27
9	Indeksi	30
10	Komentari	31
11	Zaključak	32

Poglavlje 1

Uvod

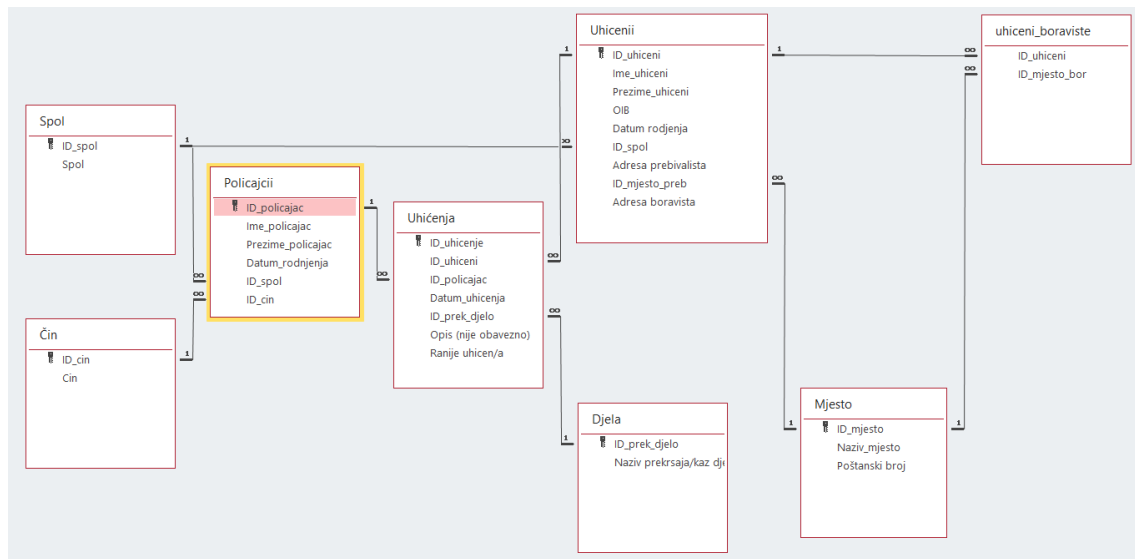
Za ovaj seminar odlučio sam izmodelirati bazu podataka policijske stanice. Razlog zašto sam se odlučio za ovu temu vrlo je jednostavan; naime, u četvrtom razredu srednje škole na nastavi informatike radili smo relacijske baze podataka u programu *MS Access* te smo u sklopu tog dijela gradiva morali napraviti projekt vlastite baze podataka. Tada sam napravio bazu podataka za policijsku stanicu te sam odlučio reciklirati ideju i opet izmodelirati bazu podataka za policijsku stanicu na ovom kolegiju. Od originalne baze iz 2023. godine ima nekoliko izmjena koje ćemo pokriti u idućim dijelovima.

1.1 *MS Access* baza podataka policijske stanice - njeni problemi i nedostatci

Koliko god Microsoftovi alati bili dobri ili dovoljni za generalnu populaciju, za bilo kakvu ozbiljniju vrstu rada potrebni su specifični alati. Primjerice, za pisanje teksta većini ljudi dovoljan je *Microsoft Word*, ali matematičarima je potreban $\text{\LaTeX}$ zbog pisanja matematičkih formula. Tako je i *Microsoft Access* možda dovoljan većini populacije za modeliranje baza podataka, ali je podatkovnim analitičarima i podatkovnim znanstvenicima potrebno nešto konkretnije od tako jednostavnog i ograničenog modela. Stoga oni većinom koriste SQL (engl. *Standard Query Language*) za modeliranje relacijskih baza podataka. Pogledajmo model baze podataka iz 2023. te ga analizirajmo.

Kao što se može vidjeti na slici 1.1, ova baza podataka ima puno problema. Prvenstveno, nije potrebna posebna tablica za čuvanje informacije o spolu. Nadalje, željeno ponašanje tablice nije bilo u skladu s ostvarenim. Željeno ponašanje bilo je da tablica mjesto ima dvije veze stranog ključa u tablici Uhićeni (jedna za prebivalište i jedna za boravište), što nije bilo moguće za ostvariti u *MS Accessu*. Osim toga, model je trebalo proširiti zbog uvjeta projekta, ali i s obzirom na razinu obrazovanja i kolegija. Entiteti kojima sam je htio proširiti su:

- odjel u kojem policajac radi,
- vozilo koje policajac koristi,
- školu koju je policajac završio

Slika 1.1: Model originale *MS Access* baze podataka

Također, htio sam istestirati može li se nekako atribut "ranije uhićen/a" modificirati tako da se automatski postavi, bez ljudske tj. ručne provjere u bazi podataka. Više o modelu implementacije svega navedenog u idućem poglavlju.

Poglavlje 2

MEV i relacijski model baze podataka

Koliko god se ovaj korak izrade MEV modela i relacijskog modela baze podataka nekima činio bespotrebnim u njenoj izradi, ne znači da je samim time i manje važan; čak bih rekao da je, poučen novim iskustvom, taj korak upravo i najbitniji korak u izradi baze podataka. Na tim modelima uvijek se može provjeriti nazive atributa u tablici ili način na koji su povezani, što uvelike pomaže pri pisanju kompleksnih upita.

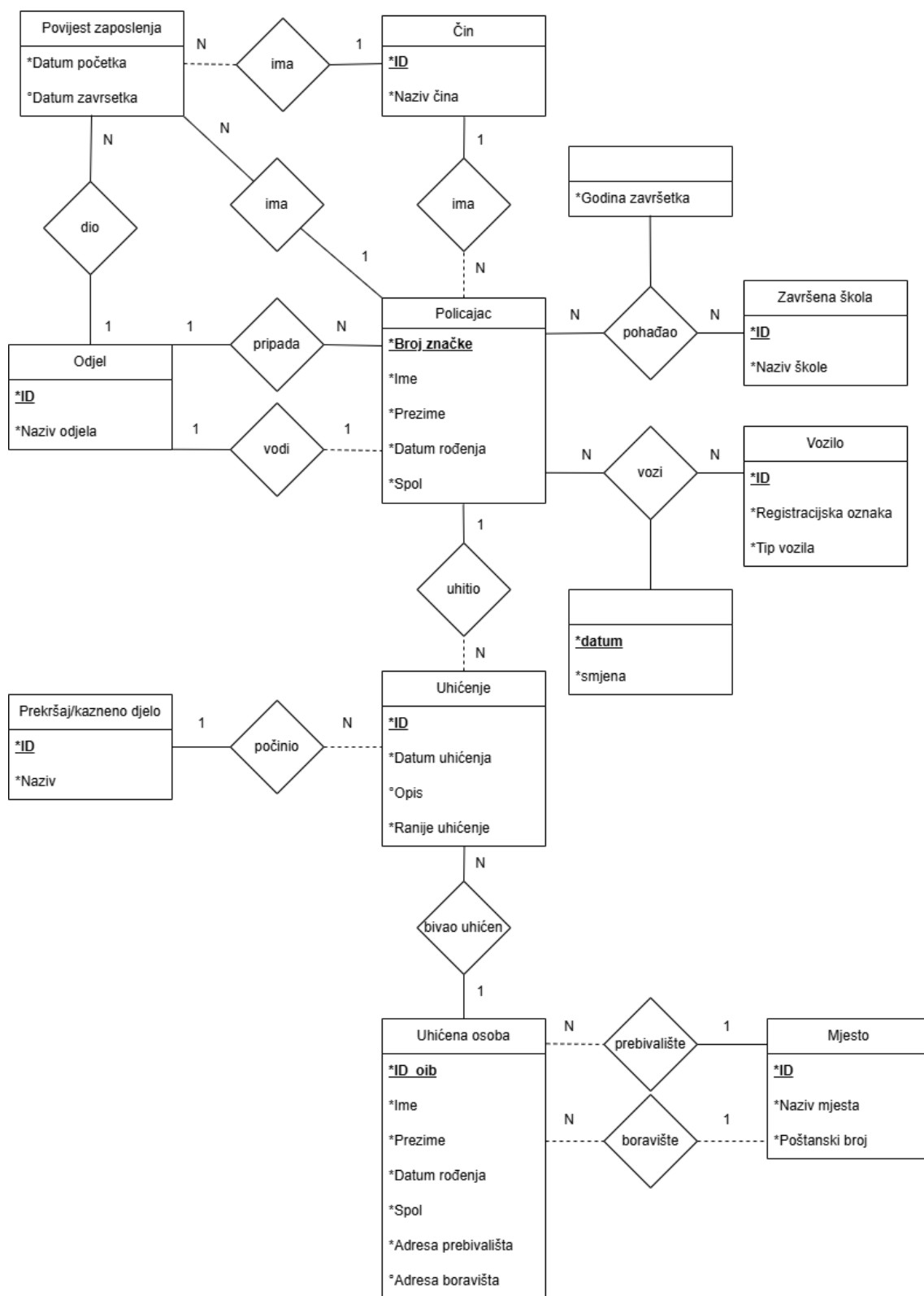
2.1 MEV model

Model se vrti oko tri glavna entiteta; to su policajac, uhićena osoba i uhićenje (uhićenje vrši policajac nad uhićenom osobom). Ti "glavni" entiteti dodatno su povezani tj. opisani dodatnim entitetima. Policajca sam "obogatio" činom koji drži (jedan čin može pripadati više policajaca, svaki policajac mora imati točno jedan čin u danom trenutku), odjelom kojem pripada (odjel može imati točno jednog voditelja iz reda policajaca, svaki policajac mora pripadati točno jednom odjelu; nadalje, svaki policajac mora pripadati točno jednom odjelu i svaki odjel mora imati barem jednog policajca), školom koju je završio (svaki policajac može završiti više škola, svaku školu moglo je pohađati više policajaca, razlikuju se po godini završetka), vozilom koje je koristio (svaki policajac može se koristiti više vozila, ali barem jednim mora, također svako vozilo mora biti voženo od strane barem jednog policajca, razlikovno svojstvo između njih je, osim policajca i vozila, i datum na koji je to vozilo bilo voženo) i povijesti zaposlenja u toj policijskoj stanici (detektira svaku promjenu odjela ili čina, više o tome poslije). Nadalje, uhićenoj osobi pripada entitet mjesto koji osobi daje informaciju o mjestu prebivališta i boravišta, npr. daje informaciju o poštanskom broju mjesta. Iako je nekakvo popularno mišljenje da poštanski broj može na jedinstven način definirati mjesto, postoje mjesta u Republici Hrvatskoj koja dijele isti poštanski broj (npr. Valpovo i Ladimirevci). Iz svakog mjesta može poteći više uhićenih osoba, ali svaka osoba mora imati točno jednu adresu prebivališta, ali može imati više adresa boravišta. Na kraju, uhićenje vrši policajac (svaki policajac može imati više uhićenja, svako uhićenje pripada točno jednom policajcu) nad uhićenom osobom (svaka osoba može biti uhićena više puta, ali barem

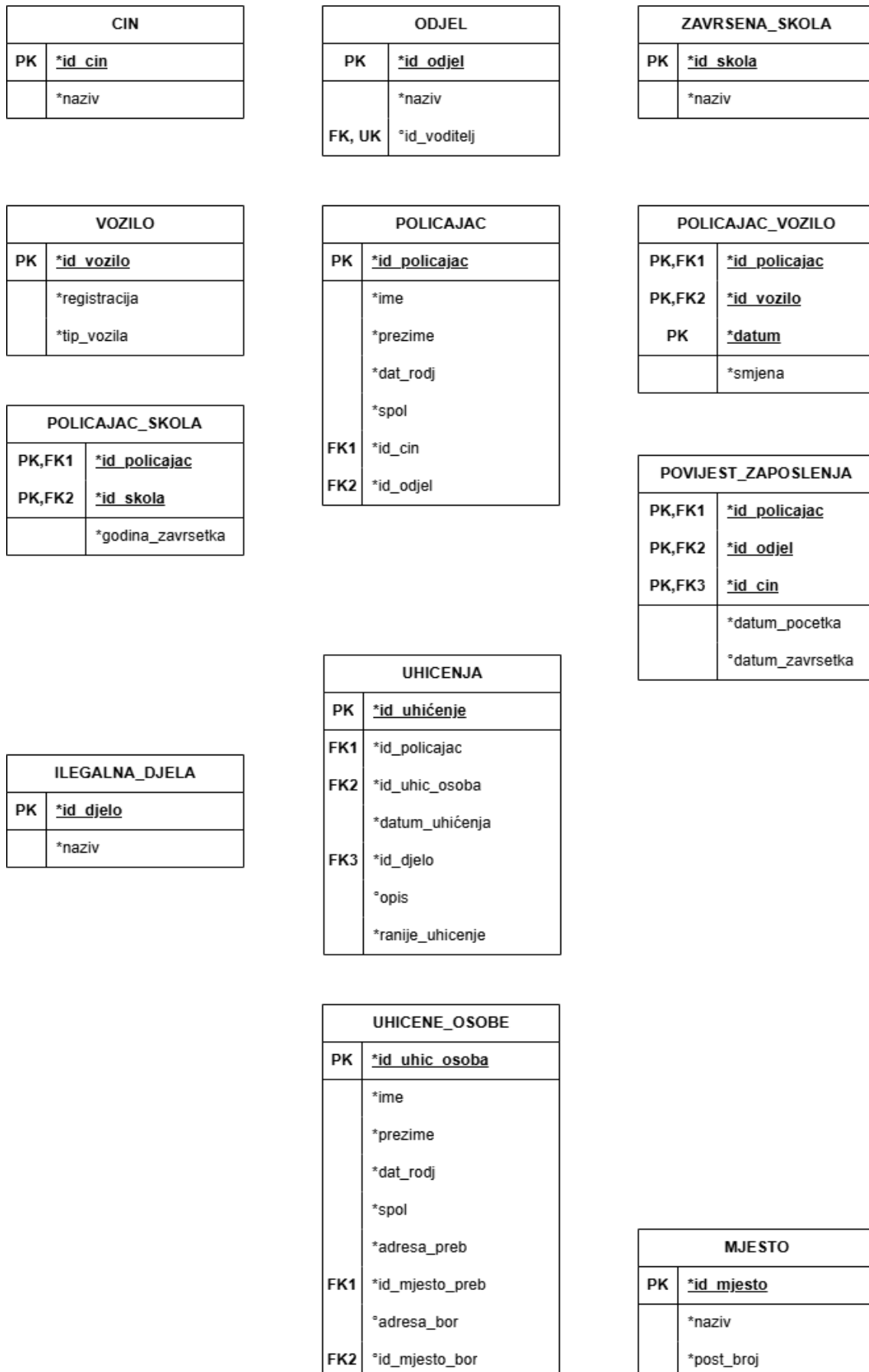
jednom da bi bila u bazi, dok svakom uhićenju pripada točno jedna osoba) te s tim uhićenjem mora biti povezano kazneno djelo ili prekršaj (svako uhićenje ima točno jedno zlodjelo, dok svako zlodjelo može biti počinjeno u više uhićenja). MEV model nalazi se na slici 2.1.

2.2 Relacijski model

Prebacivanje iz MEV modela u relacijski nije bilo prezahtjevno. Relacijski model može se vidjeti na slici 2.2. Većina ostvarenih veza je 1:N, stoga je njihova implementacija bila jednostavna (zadavanje stranog ključa u tablici gdje je N), dok je mandatornost atributa od ranije zadana MEV modelom. Ovdje bih se jedino htio dodatno napomenuti tablice `policajac_skola` i `policajac_vozilo` koje služe za ostvarivanje veza N:N. Tablica `policajac_vozilo` i pripadni entitet u MEV modelu imaju atribut `smjena` koji samo drži dodatnu informaciju o tome u kojoj je smjeni koji policajac na dani datum koristio dano vozilo. Nadalje, u ovoj bazi podataka postoji veza 1:1 i nalazi se između tablica `odjel` i `policajac`, vezano za voditelja odjela. Normalno je atribut `id_voditelj` definiran kao strani ključ koji se referira na stupac `id_policajac` u tablici `policajac`. Ali, da bi osigurali jedinstvenost, potrebno je definirati da je atribut jedinstven tj. **UNIQUE** (više o tome kod kreiranja tablica). Taj je atribut opcionalan iz dva razloga. Prvi je taj što je moguće da odjel nema šefa (V.D. se ne broji u ovom slučaju), a drugi je što je tako definirano MEV modelom. Ovo će nam naknadno olakšati definiranje tablica i unos u njih. Više tablica ima primarni ključ definiran na više stupaca, poput `policajac_skola`, `policajac_vozilo` i `povijest_zaposlenja`. One su jedinstveno definirane i u MEV modelu s više atributa, stoga se i njihov primarni ključ sastoji od više stupaca u relacijskom modelu.



Slika 2.1: MEV model baze podataka



Slika 2.2: Relacijski model baze podataka

Poglavlje 3

Kreiranje tablica

Sama izrada tablica direktno je slijedila relacijski model. Nema se tu što puno za nadodati jer većina tablica svojom kreacijom slijedi relacijski model. U danom primjeru na slici 4.1 možemo vidjeti kreiranje tablice `mjesto` i kreiranje tablice `uhicene_osobe`.

```
CREATE TABLE mesto(  
    id_mjesto SERIAL PRIMARY KEY,  
    naziv VARCHAR(50) NOT NULL,  
    post_broj VARCHAR(5) NOT NULL  
);  
  
CREATE TABLE uhicene_osobe(  
    id_uhic_osoba VARCHAR(11) PRIMARY KEY, -- oib  
    ime VARCHAR(20) NOT NULL,  
    prezime VARCHAR(30) NOT NULL,  
    dat_rodj DATE NOT NULL,  
    spol VARCHAR(1) NOT NULL CONSTRAINT spol_chk CHECK (spol in ('M', 'F')),  
    adresa_preb VARCHAR(100) NOT NULL,  
    id_mjesto_preb INTEGER NOT NULL CONSTRAINT mesto_preb_fk REFERENCES mesto(id_mjesto),  
    adresa_bor VARCHAR(100),  
    id_mjesto_bor INTEGER CONSTRAINT mesto_bor_fk REFERENCES mesto(id_mjesto)  
);
```

Slika 3.1: Kreiranje tablica `mjesto` i `uhicene_osobe`

Primijetimo da su atributi `id_mjesto_preb` i `id_mjesto_bor` zadani kao strani ključevi koji se referenciraju na tablicu `mjesto`. Potonji atribut, kao i njemu pripadna adresa, mogu poprimiti i NULL vrijednosti zbog toga što je moguće da osoba nema mjesto boravišta. U tom slučaju postoji okidač (TRIGGER) koji će se pokrenuti i automatski postaviti te vrijednosti na vrijednosti za prebivalište (više o tome pod temom okidača).

Nadalje, pogledajmo atribut `spol` u tablici `uhicene_osobe`. Dodali smo uvjet provjere je li spol zadan slovom M ili F, stoga ne prihvaća unose bilo kojih drugih alfanumeričkih znakova, osim ta dva (više o tome pod temom uvjeta). Na kraju, htio bih napomenuti da primarni ključ tablice `uhicene_osobe` predstavlja OIB te osobe. OIB je niz brojeva od 11 znamenki koji na jedinstven način opisuje svaku državljanku ili državljanina Republike Hrvatske. Stoga je to vrlo pogodan atribut za primarni ključ. U slučaju ove baze podataka, spremljen je kao niz od 11 alfanumeričkih

znakova zato što bi spremanje tog podatka kao broja zauzimalo previše memorije, a moguće je da i pređe gornju granicu koju tip `INTEGER` može poprimiti.

Na sličan način gore navedenom načinu definirane su tablice `ilegalna_djela`, `uhicenja`, `cin`, `zavrsena_skola` i `vozilo`. Na preostale tablice htio bih se referirati u sljedećem dijelu.

Prvo, referirajmo se na tablice `policajac` i `vozilo` te tablicu `policajac_vozilo`. Prema MEV modelu, veza između entiteta `policajac` i `vozilo` je N:N, stoga je potrebna dodatna tablica koja ih povezuje. U tom slučaju to je tablica `policajac_vozilo`. Kreiranje danih tablica dane su slikama 3.2 i 3.3.

```
CREATE TABLE vozilo (
  id_vozilo VARCHAR(20) PRIMARY KEY, -- broj šasije
  registracija VARCHAR(7) NOT NULL,
  tip_vozila VARCHAR(20) NOT NULL CONSTRAINT tip_vozila_check CHECK (tip_vozila IN ('automobil', 'motor', 'marica', 'kombi', 'brod'))
);

CREATE TABLE policajac (
  id_policajac VARCHAR(9) PRIMARY KEY,
  ime VARCHAR(20) NOT NULL,
  prezime VARCHAR(30) NOT NULL,
  dat_rodj DATE NOT NULL,
  spol VARCHAR(1) NOT NULL CONSTRAINT spol_chk CHECK (spol IN ('M', 'F')),
  id_cin INTEGER NOT NULL CONSTRAINT cin_fk REFERENCES cin(id_cin),
  id_odjel INTEGER NOT NULL
);
```

Slika 3.2: Kreiranje tablica `vozilo` i `policajac`

```
CREATE TABLE policajac_vozilo(
  id_policajac VARCHAR(9) NOT NULL CONSTRAINT policajac_vozilo_fk REFERENCES policajac(id_policajac),
  id_vozilo VARCHAR(20) NOT NULL CONSTRAINT vozilo_fk REFERENCES vozilo(id_vozilo),
  datum DATE NOT NULL,
  smjena INTEGER NOT NULL CONSTRAINT smjena_ck CHECK (smjena IN (1, 2, 3)),
  PRIMARY KEY (id_policajac, id_vozilo, datum)
);
```

Slika 3.3: Kreiranje tablice `policajac_vozilo`

Primijetimo da je u tablici `policajac_vozilo` primarni ključ dan na kraju tablice i da je dan kao skup podataka kojeg čine `id_policajac`, `id_vozilo` i `datum`. Iz očitih razloga možemo reći da `policajac`, `vozilo` koje vozi i dan na kojeg ga vozi jedinstveno definiraju jedinstvenu redak u veznoj tablici. U tablici `policajac_vozilo` nalazi se još atribut `smjena` sa svojim `CHECK` uvjetom na broj smjene, koji opisuje u kojoj smjeni je dani `policajac` na dani `datum` koristio dano `vozilo`. Na analogan način riješili smo problem između tablica `policajac` i `zavrsena_skola`. Definirali smo novu tablicu `policajac_skola` koja jedinstveno određuje `policajca` i školu koju je završio, zajedno s godinom kada je danu školu završio.

```
CREATE TABLE odjel(
  id_odjel SERIAL PRIMARY KEY,
  naziv VARCHAR(50) NOT NULL,
  id_voditelj VARCHAR(9) CONSTRAINT voditelj_fk REFERENCES policajac(id_policajac)
);
```

Slika 3.4: Kreiranje tablice `odjel`

Nadalje, htio bih se referirati na stvaranje tablica `odjel` i `policajac`, tj. na njihovo povezivanje. Tablicu `policajac` možete vidjeti na slici 3.2, dok je tablica `odjel` odjel vidljiva na slici 3.4. Primijetimo da atributu `id_voditelj` u tablici `odjel` nije dan `UNIQUE` uvjet i da atributu `id_odjel` u tablici `policajac` nije dan uvjet stranog ključa. To je zato što je kreiranje tablice `policajac` obavljeno prije kreiranja tablice `odjel` i zato što nije moguće, barem ne jednostavno, definirati više od jednog `CONSTRAINT`-a pri kreiranju tablice. Stoga ta dva uvjeta moramo kasnije nadodati. Uvjeti su nadodani na način prikazanom na slici 3.5.

```
ALTER TABLE odjel
ADD CONSTRAINT voditelj_uq UNIQUE (id_voditelj);

ALTER TABLE policajac
ADD CONSTRAINT odjel_fk FOREIGN KEY (id_odjel) REFERENCES odjel(id_odjel);
```

Slika 3.5: Izmjena tablica `odjel` i `policajac` radi postizanja uvjeta relacijskog modela

Na samom kraju htio bih se referirati na to da postoje različiti okidači, funkcije i procedure definirane na tablicama. Njih ću detaljnije obraditi u poglavljima koje su za njih predviđeni, a spomenut ću ih kada za njih dođe vrijeme u idućim poglavljima. Trenutno bih htio priložiti početak skripte u kojem je napisan način brisanja svih tablica pomoću ključne riječi `DROP`. To se može vidjeti na slici 3.6.

```
DROP TABLE IF EXISTS uhicenja;
DROP TABLE IF EXISTS uhicene_osobe;
DROP TABLE IF EXISTS ilegalna_djela;
DROP TABLE IF EXISTS mjesto;
DROP TABLE IF EXISTS povijest_zaposlenja;
DROP TABLE IF EXISTS policajac_vozilo;
DROP TABLE IF EXISTS policajac_skola;
DROP TABLE IF EXISTS policajac CASCADE;
DROP TABLE IF EXISTS vozilo;
DROP TABLE IF EXISTS završena_skola;
DROP TABLE IF EXISTS odjel;
DROP TABLE IF EXISTS cin;
```

Slika 3.6: Brisanje svih tablica na početku skripte

Tablica `policajac` mora se brisati kaskadno jer je povezana s tablicom `odjel` koja je povezana s tablicom `policajac`.

Poglavlje 4

Unosi u tablice

Većina unosa u tablicu bila je analogna unosima u tablicu `mjesto`, prikazano, na slici 4.1.

```
INSERT INTO mjesto(naziv, post_broj)
VALUES
('Osijek', '31000'),
('Brijest', '31107'),
('Briješće', '31107'),
('Josipovac', '31107'),
('Klisa', '31000'),
('Nemetin', '31000'),
('Podravlje', '31000'),
('Sarvaš', '31000'),
('Tvrđavica', '31000'),
('Tenja', '31207'),
('Višnjevac', '31220'),
('Bilje', '31327'),
('Zagreb', '10000'),
('Split', '21000');
```

Slika 4.1: Unosi u tablicu `mjesto`

Na analogan način podatci su se unosili u tablice `cin`, `zavrsena_skola`, `vozilo`, i `ilegalna_djela`. Pogledajmo primjer unosa u tablicu `policajac` koja prima i dva strana ključa, `id_cin` i `id_odjel` na slici 4.2. Primijetimo da je sve skoro analogno, samo na kraju (zadnja 2 atributa) unosimo čudne brojeve koji predstavljaju strane ključeve s kojima se spajamo u baznim tablicama. Analogno se ovom načinu unosi i u tablice `uhicene_osobe` i `uhicenja`, s njihovim primjerenim stranim ključevima.

Za primjer unosa u tablicu koja ostvaruje vezu N:N pogledajmo primjer unosa u tablicu `policajac_vozilo`. Kao što vidimo, unijeli smo potrebne podatke za rad nad tablicama i ostvarivanje veze N:N i još dodatan podatak o smjeni u kojoj se auto korisito. Primjer je vidljiv na slici 4.3. Analogno se unosi u tablicu `policajac_skola`.

Na kraju, htio bih se referirati na unos u dvije specifične tablice, tablice `odjel` i `povijest_zaposlenja`. Kao što se može vidjeti na slici 4.4, odjele smo unijeli bez

```
INSERT INTO policajac (id_policajac, ime, prezime, dat_rodj, spol, id_cin, id_odjel)
VALUES
('5362-4324', 'Ivan', 'Primorac', '06.11.1956.', 'M', 4, 4),
('4325-5287', 'Leon', 'Lučić', '25.07.1982.', 'M', 5, 4),
('3422-8732', 'Anja', 'Morić', '31.03.1983.', 'F', 5, 4),
('2432-5328', 'Vesna', 'Kozar', '12.01.1987.', 'F', 5, 4),
('0001-0001', 'Berislav', 'Plenković', '01.01.2006.', 'M', 1, 6),
('7342-2738', 'Božidar', 'Šimunović', '25.12.1972.', 'M', 3, 2),
('5463-1463', 'Marko', 'Popović', '27.04.2004.', 'M', 8, 3),
('5321-2436', 'Marko', 'Kovačić', '17.01.2005.', 'M', 8, 1),
('4322-4713', 'Ivona', 'Širić', '10.11.2004.', 'F', 5, 1),
('4327-4128', 'Ivan', 'Ivanić', '10.04.1995.', 'M', 4, 3);
```

Slika 4.2: Unosi u tablicu policajac

```
INSERT INTO policajac_vozilo (id_policajac, id_vozilo, datum, smjena)
VALUES
('4325-5287', 'TMBOC9NE720216232', '11.09.2011.', 1),
('2432-5328', 'TMBOC9NE720216232', '11.09.2011.', 2),
('5321-2436', 'TMBOC9NE720197532', '24.10.2023.', 1),
('4322-4713', 'WBAGRE8G820195178', '07.12.2023.', 3),
('5362-4324', 'WVWCCW98420203328', '10.11.2021.', 3),
('5463-1463', 'WVWCVFSD820178429', '13.06.2024.', 2);
```

Slika 4.3: Unosi u tablicu policajac_vozilo

atributa `id_voditelj`. To smo napravili zato što smo odjele unijeli prije policajaca, ali i zbog toga što je `id_voditelj` neobavezan atribut tj. može (barem privremeno) poprimiti i NULL vrijednosti.

```
INSERT INTO odjel (naziv)
VALUES
('Prometna policija'),
('Interna policija'),
('Interventa policija'),
('Kriminalistika'),
('Granična policija'),
('Uprava');
```

Slika 4.4: Unosi u tablicu odjel

Navedeno ponašanje popravili smo ažuriranjem odjela na primjerene šefove tih odjela kao što je prikazano slikom 4.5.

Na samom kraju, pokrio bih još i ponašanje tablice `povijest_zaposlenja`, tj. način unosa u tu tablicu. Tu sam tablicu zamislio kao tablicu koja "automatski" u sebe unosi podatke tj. nije potrebno ručno u nju unositi podatke (naravno, neki povijesni podatci moraju se ručno unositi). Glavna ideja koju sam imao je da, osim običnih INSERT-a u tablicu `povijest_zaposlenja` (koji služe za pohranjivanje podataka iz neke npr. stare knjige s podacima u novu bazu podataka) da još

```
UPDATE odjel
SET id_voditelj = '4322-4713'
WHERE id_odjel = 1;

UPDATE odjel
SET id_voditelj = '7342-2738'
WHERE id_odjel = 2;

UPDATE odjel
SET id_voditelj = '4327-4128'
WHERE id_odjel = 3;

UPDATE odjel
SET id_voditelj = '5362-4324'
WHERE id_odjel = 4;

UPDATE odjel
SET id_voditelj = '0001-0001'
WHERE id_odjel = 6;
```

Slika 4.5: Ažuriranja u tablici `odjel` radi postavljanja šefova

dodam proceduru `dodaj_u_povijest` koja dodaje podatke od policajca u tablicu s nekim datumom početka (ideja je da se svi "novi" policajci automatski dodaju u `povijest_zaposlenja` pri unosu preko te procedure). Naravno, neki raniji unosi uneseni su ručno u tablicu, ali ideja koju sam imao pri ovoj zamisli je da se to automatizira u nekakvoj aplikaciji sa UI (*user interface*, ne umjetna inteligencija) te da se pri svakom novom unosu u tablicu pokrene i ta procedura. Nisam je htio staviti kao okidač s današnjim datumom jer bi se to onda pokrenulo i za do sada unesene policajce, a to bi bilo neželjno ponašanje. Više o proceduri `dodaj_u_povijest` u poglavlju primijerenom za tu temu. Navedeni inserti vidljivi su na slici 4.6

```
INSERT INTO povijest_zaposlenja(id_policajac, id_odjel, id_cin, datum_pocetka, datum_zavrsetka)
VALUES
('5362-4324', 4, 5, '01.01.2000.', '01.02.2007.'),
('4325-5287', 4, 8, '01.04.2004.', '01.02.2007.'),
('3422-8732', 4, 8, '01.03.2003.', '01.02.2007.'),
('2432-5328', 4, 8, '01.05.2005.', '01.11.2007.'),
('7342-2738', 3, 8, '01.02.2003.', '01.10.2007.'),
('7342-2738', 2, 6, '01.10.2007.', '07.11.2013.'),
('4327-4128', 3, 8, '01.01.2015.', '15.07.2018.');
```

```
INSERT INTO povijest_zaposlenja(id_policajac, id_odjel, id_cin, datum_pocetka)
VALUES
('5362-4324', 4, 4, '01.02.2007.'),
('4325-5287', 4, 5, '01.02.2007.'),
('3422-8732', 4, 5, '01.02.2007.'),
('2432-5328', 4, 5, '01.11.2007.'),
('7342-2738', 2, 3, '07.11.2013.'),
('4327-4128', 3, 4, '15.07.2018.');
```

```
CALL dodaj_u_povijest('0001-0001', '01.01.2025.');
```

```
CALL dodaj_u_povijest('5463-1463', '02.10.2024.');
```

```
CALL dodaj_u_povijest('5321-2436', '02.10.2024.');
```

```
CALL dodaj_u_povijest('4322-4713', '02.10.2024.');
```

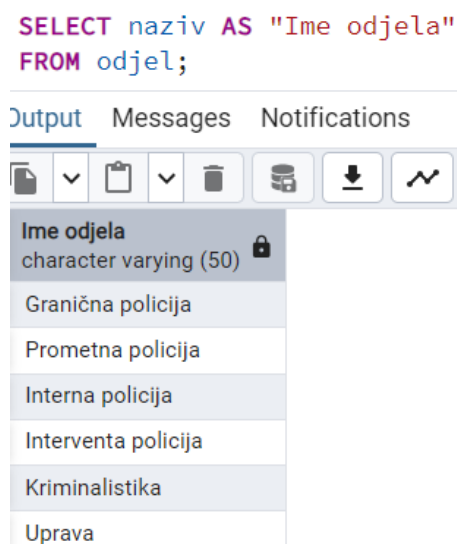
Slika 4.6: Unosi u tablicu `povijest_zaposlenja`

Poglavlje 5

Upiti

5.1 Jednostavni upiti

Jednostavni upiti su upiti nad jednom tablicom. U prilogima 5.1, 5.2, 5.3, 5.4 i 5.5 nalazi se pet jednostavnih upita nad tablicama baze podataka, zajedno s njihovim rezultatima.



Slika 5.1: Upit : svi odjeli koji se nalaze u policijskoj stanici


```
SELECT post_broj || ' ' || naziv AS "Poštanski broj i mjesto"
FROM mjesto;
```

Output Messages Notifications	
📄 📋 🗑️ 🔍 ⬇️ 📶 SQL	
Poštanski broj i mjesto	text
31000 Osijek	
31107 Brijest	
31107 Briješće	
31107 Josipovac	
31000 Klisa	
31000 Nemetin	
31000 Podravlje	
31000 Sarvaš	
31000 Tvrdavica	
31207 Tenja	
31220 Višnjevac	
31327 Bilje	
10000 Zagreb	
21000 Split	

Slika 5.2: Upit : sva mjesta s poštanskim brojevima

```
SELECT id_uhic_osoba AS "OIB", ime || ' ' || prezime AS "Ime i prezime"
FROM uhicene_osobe;
```

Output Messages Notifications	
📄 📋 🗑️ 🔍 ⬇️ 📶 SQL	
OIB	Ime i prezime
character varying (11)	text
64283746243	Tatjana Stranjak
42387763428	Ivo Sanader
56132878734	Jadranko Leši...
46328764183	Mihaela Šego
99327017582	Ana Ivanović

Slika 5.3: Upit : OIB, ime i prezime svih uhićenih osoba

```
SELECT id_policajac, ime || ' ' || prezime AS "Ime i prezime"
FROM policajac
WHERE spol = 'F';
```

Output Messages Notifications	
📄 📋 🗑️ 🔍 ⬇️ 📶 SQL	
id_policajac	Ime i prezime
[PK] character varying (9)	text
3422-8732	Anja Morić
2432-5328	Vesna Kozar
4322-4713	Ivona Širić

Slika 5.4: Upit : sve djelatnice policije

```
SELECT *
FROM policajac
WHERE id_odjel = 4;
```

Output Messages Notifications

id_policajac	ime	prezime	dat_rodj	spol	id_cin	id_odjel
[PK] character varying (9)	character varying (20)	character varying (30)	date	character varying (1)	integer	integer
5362-4324	Ivan	Primorac	1956-11-06	M	4	4
4325-5287	Leon	Lučić	1982-07-25	M	5	4
3422-8732	Anja	Morić	1983-03-31	F	5	4
2432-5328	Vesna	Kozar	1987-01-12	F	5	4

Slika 5.5: Upit : svi podatci o djelatnicima odjela kojem je `id_odjel = 4`

5.2 Upiti nad više tablica

Ovi su upiti poput jednostavnih, osim što koriste više tablica. Tablice su povezane JOINovima. Upiti su vidljivi na slikama 5.6, 5.7, 5.8, 5.9, 5.10. Prvi rezultat upita 5.6 napravljen je kao *easter egg* na situaciju koja se zapravo dogodila i koja je spomenuta u originalnoj bazi podataka. Više o tome pitajte uključenog kolegu.

```
SELECT uo.ime || ' ' || uo.prezime AS "Ime i prezime", uo.id_uhic_osoba AS "OIB", u.datum_uhicenja, u.opis
FROM uhicenja u INNER JOIN uhicene_osobe uo USING (id_uhic_osoba);
```

Output Messages Notifications

Ime i prezime	OIB	datum_uhicenja	opis
text	character varying (11)	date	text
Tatjana Stranjak	64283746243	2022-11-27	Optužena pokušala prebiti učenika Marka Kovačića.
Ivo Sanader	42387763428	2009-11-13	U zatvor š njim!
Mihaela Šego	46328764183	2021-01-25	Optužena pucala po Novom Zagrebu.
Ana Ivanović	99327017582	2025-06-01	[null]
Jadranko Lešina	56132878734	2025-04-10	[null]

Slika 5.6: Upit : sve uhićene osobe, njihov OIB i opis okolnosti uhićenja

```
SELECT p.ime || ' ' || p.prezime AS "Ime i prezime", o.naziv, c.naziv, pz.datum_pocetka, pz.datum_zavrsetka
FROM povijest_zaposlenja pz INNER JOIN odjel o USING (id_odjel)
INNER JOIN cin c USING (id_cin)
INNER JOIN policajac p USING (id_policajac)
WHERE id_policajac = '0001-0001';
```

Output Messages Notifications

Ime i prezime	naziv	naziv	datum_pocetka	datum_zavrsetka
text	character varying (50)	character varying (50)	date	date
Berislav Plenković	Uprava	Načelnik policije	2025-01-01	[null]

Slika 5.7: Upit : smisljena povijest zaposlenja policajca čiji je ID = '0001-0001'

```
SELECT uo.ime || ' ' || uo.prezime AS "Ime i prezime"
FROM uhicene_osobe uo INNER JOIN mjesto m
ON uo.id_mjesto_preb = m.id_mjesto
INNER JOIN mjesto m2
ON uo.id_mjesto_bor = m2.id_mjesto
WHERE NOT (uo.id_mjesto_preb = uo.id_mjesto_bor AND uo.adresa_preb = uo.adresa_bor);
```

Output Messages Notifications

Ime i prezime text
Mihaela Šego
Ana Ivanović

Slika 5.8: Upit : uhićene osobe čije je boravište različito od prebivališta

```
SELECT p.ime || ' ' || p.prezime AS "Ime i prezime", o.naziv AS "Ime odjela"
FROM odjel o INNER JOIN policajac p
ON o.id_voditelj = p.id_policajac;
```

Output Messages Notifications

Ime i prezime text	Ime odjela character varying (50)
Ivona Širić	Prometna policija
Božidar Šimunović	Interna policija
Ivan Ivanić	Interventa policija
Ivan Primorac	Kriminalistika
Berislav Plenković	Uprava

Slika 5.9: Upit : imena policajaca i odjela koje vode

```
SELECT DISTINCT idj.naziv
FROM ilegalna_djela idj INNER JOIN uhicenja USING (id_djelo);
```

Output Messages Notifications

naziv character varying (50)
Distribucija opojnih sredstava
Primanje mita
Remećenje javnog reda i mira
Pokušaj ubojstva
Vandalizam

Slika 5.10: Upit : ilegalna djela zbog kojih su ljudi do sada bili uhićeni u policijskoj stanici

5.3 Upiti s agregirajućim funkcijama

Agregirajuće funkcije su funkcije koje se mogu koristiti u upitima da se npr. pronađe minimum, maksimum, prosjek ili da se prebroje redovi po danom uvjetu. Primjeri agregirajućih funkcija su MIN, MAX, AVG, COUNT. Moraju se koristiti uz GROUP BY da znamo po čemu računamo. Agregirajuće upite i njihove rezultate možemo vidjeti na slikama 5.11, 5.12, 5.13, 5.14, 5.15.

```
SELECT spol, COUNT(*)
FROM policajac
GROUP BY spol;
```

Output Messages Notifications

spol	count
character varying (1)	bigint
M	7
F	3

Slika 5.11: Upit : broj policajaca po spolu

```
SELECT p.ime || ' ' || p.prezime AS "Ime i prezime", MAX(current_date - pz.datum_pocetka)
FROM povijest_zaposlenja pz INNER JOIN policajac p USING (id_policajac)
GROUP BY "Ime i prezime";
```

Output Messages Notifications

Ime i prezime	max
text	integer
Ivona Širić	268
Ivan Ivanić	3830
Božidar Šimuno...	8182
Berislav Plenković	177
Marko Popović	268
Leon Lučić	7757
Ivan Primorac	9309
Anja Morić	8154
Vesna Kozar	7362
Marko Kovačić	268

Slika 5.12: Upit : koliko dugo je svaki policajac zaposlen (u danima)

```
SELECT m.naziv, COUNT(*)
FROM uhicene_osobe uo INNER JOIN mjesto m
ON uo.id_mjesto_preb = m.id_mjesto
GROUP BY m.naziv;
```

Output Messages Notifications

naziv character varying (50)	count bigint
Zagreb	1
Osijek	4

Slika 5.13: Upit : broj uhićenih prema mjestu prebivališta

```
SELECT tip_vozila, COUNT(*)
FROM vozilo
GROUP BY tip_vozila
HAVING COUNT(*) > 1;
```

Output Messages Notifications

tip_vozila character varying (20)	count bigint
automobil	3
motor	2

Slika 5.14: Upit : tipovi vozila kojih u garaži ima u najmanje dva primjerka

```
SELECT v.registracija, count(*)
FROM policajac_vozilo pv INNER JOIN vozilo v USING (id_vozilo)
GROUP BY v.registracija;
```

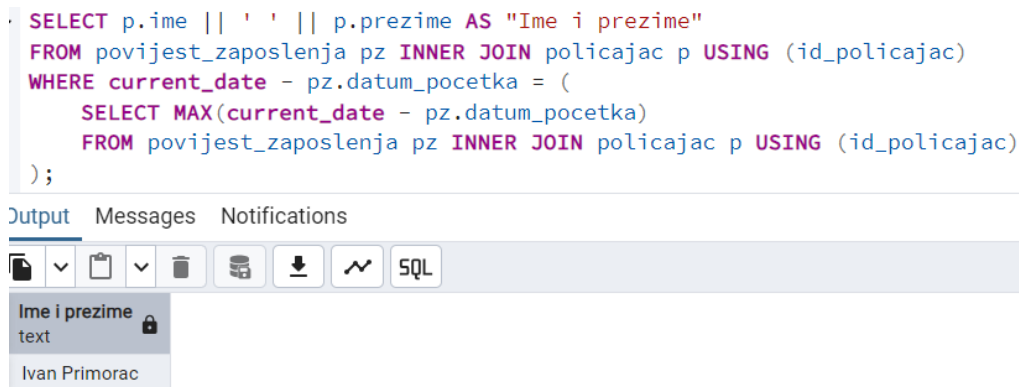
Output Messages Notifications

registracija character varying (7)	count bigint
130-129	2
050-235	1
140-150	1
060-123	1
231-428	1

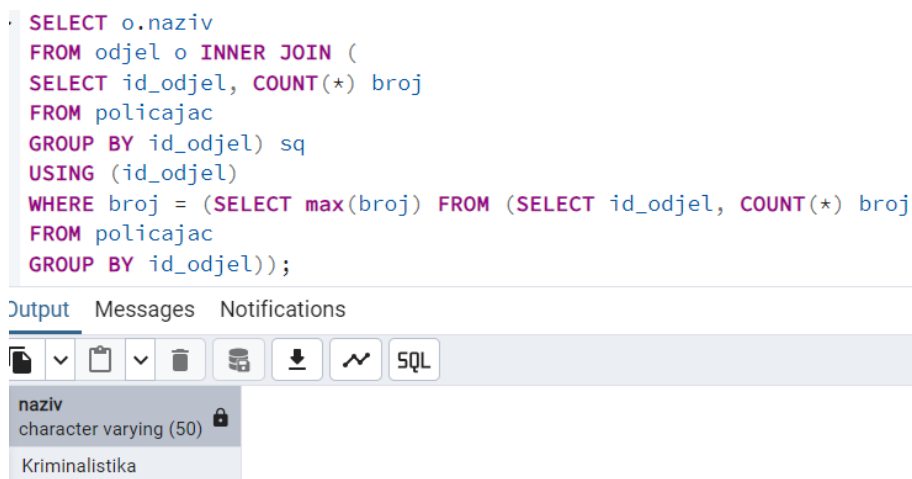
Slika 5.15: Upit : broj korištenja prema vozilu

5.4 Upiti s podupitima, ugnježdjenim upitima ili skupovnim operacijama

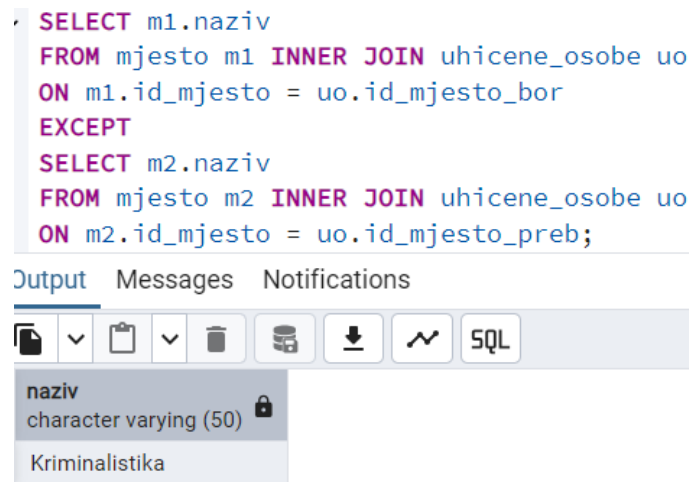
U sljedećim upitima koristit ćemo neke od ranije navedenih stvari. Ti su upiti vidljivi na slikama 5.16, 5.17, 5.18, 5.19 i 5.20.



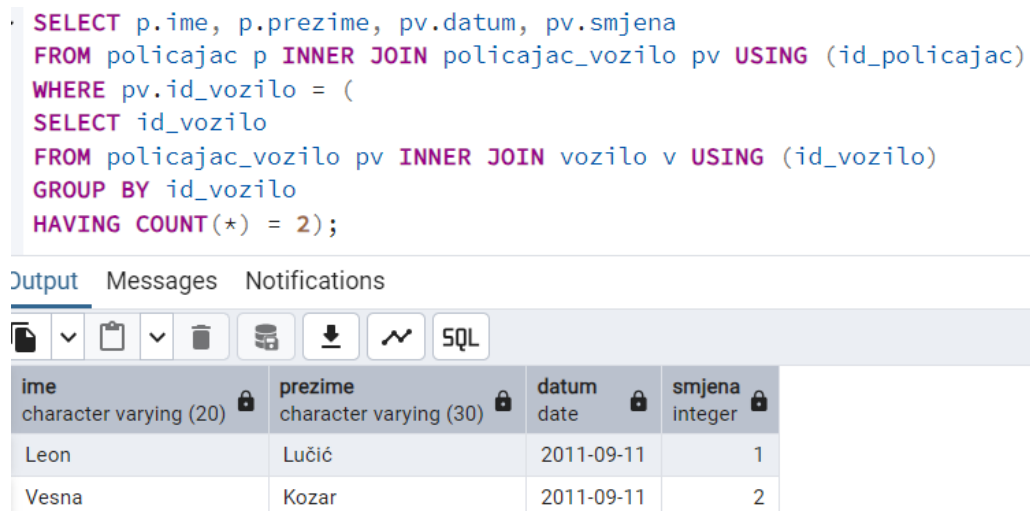
Slika 5.16: Upit : ime i prezime policajca s najduljim radnim stažom



Slika 5.17: Upit : ime odjela s najviše zaposlenika



Slika 5.18: Upit : mjesta boravišta koja nisu mjesta prebivališta



Slika 5.19: Upit : ime, prezime, datum i smjena korištenja vozila koje se koristilo točno dva puta

```
SELECT p.ime || ' ' || p.prezime AS "Ime i prezime", s.naziv as "Skola"
FROM policajac p INNER JOIN policajac_skola ps USING (id_policajac)
INNER JOIN završena_skola s USING (id_skola)
WHERE id_policajac IN (
SELECT id_policajac
FROM policajac INNER JOIN policajac_skola USING (id_policajac)
GROUP BY id_policajac
HAVING COUNT (*) > 1);
```

Output Messages Notifications

Ime i prezime text	Skola character varying (100)
Leon Lučić	Policajska škola Josip Jović, Zagreb
Leon Lučić	Veleučilište kriminalistike i javne sigurnosti, Zagreb
Berislav Plenković	Policajska škola Josip Jović, Zagreb
Berislav Plenković	Veleučilište kriminalistike i javne sigurnosti, Zagreb
Berislav Plenković	Pravni fakultet, Zagreb
Božidar Šimuno...	Pravni fakultet, Osijek
Božidar Šimuno...	Veleučilište kriminalistike i javne sigurnosti, Zagreb
Ivan Ivanić	Policajska škola Josip Jović, Zagreb
Ivan Ivanić	Veleučiliste Lavoslav Ružička, Vukovar

Slika 5.20: Upit : imena i prezimena policajaca sa nazivima završenih škola onih policajaca koji su završili barem dvije škole

Poglavlje 6

Uvjeti i zadane vrijednosti

6.1 Uvjeti

Neke vrijednosti sam pri kreiranju stavio na provjeru zadovoljala li unos u taj stupac neki uvjet. U SQL-u to se postiže stavljanjem CONSTRAINTa imena CHECK na neki od stupaca u kojem želimo izvršavati promjenu. Na slici 6.1 vidimo da u tablicama policajac i vozilo imamo uvjete provjere na dopuštenu vrstu vozila i na spol policajca. U tablici policajac_vozilo, vidljivo na slici 6.2, imamo uvjet provjere na smjenu u kojoj se vozilo koristilo (1, 2, 3 predstavljaju prvu, drugu i treću smjenu).

```
CREATE TABLE vozilo (  
  id_vozilo VARCHAR(20) PRIMARY KEY, -- broj šasije  
  registracija VARCHAR(7) NOT NULL,  
  tip_vozila VARCHAR(20) NOT NULL CONSTRAINT tip_vozila_check CHECK (tip_vozila IN ('automobil', 'motor', 'marica', 'kombi', 'brod'))  
);  
  
CREATE TABLE policajac (  
  id_policajac VARCHAR(9) PRIMARY KEY,  
  ime VARCHAR(20) NOT NULL,  
  prezime VARCHAR(30) NOT NULL,  
  dat_rodj DATE NOT NULL,  
  spol VARCHAR(1) NOT NULL CONSTRAINT spol_chk CHECK (spol IN ('M', 'F')),  
  id_cin INTEGER NOT NULL CONSTRAINT cin_fk REFERENCES cin(id_cin),  
  id_odjel INTEGER NOT NULL  
);
```

Slika 6.1: CHECK uvjeti na tablicama policajac i vozilo

```
CREATE TABLE policajac_vozilo(  
  id_policajac VARCHAR(9) NOT NULL CONSTRAINT policajac_vozilo_fk REFERENCES policajac(id_policajac),  
  id_vozilo VARCHAR(20) NOT NULL CONSTRAINT vozilo_fk REFERENCES vozilo(id_vozilo),  
  datum DATE NOT NULL,  
  smjena INTEGER NOT NULL CONSTRAINT smjena_ck CHECK (smjena IN (1, 2, 3)),  
  PRIMARY KEY (id_policajac, id_vozilo, datum)  
);
```

Slika 6.2: CHECK uvjet u tablici policajac_vozilo

```
CREATE TABLE policajac_skola(
  id_policajac VARCHAR(9) NOT NULL CONSTRAINT policajac_fk REFERENCES policajac(id_policajac),
  id_skola INTEGER NOT NULL CONSTRAINT skola_fk REFERENCES zavrшена_skola(id_skola),
  godina_zavrsetka INTEGER NOT NULL CONSTRAINT godina_chk CHECK (godina_zavrsetka > 1900),
  PRIMARY KEY (id_policajac, id_skola)
);
```

Slika 6.3: CHECK uvjet u tablici policajac_skola

```
CREATE TABLE uhicene_osobe(
  id_uhic_osoba VARCHAR(11) PRIMARY KEY, -- oib
  ime VARCHAR(20) NOT NULL,
  prezime VARCHAR(30) NOT NULL,
  dat_rodj DATE NOT NULL,
  spol VARCHAR(1) NOT NULL CONSTRAINT spol_chk CHECK (spol in ('M', 'F')),
  adresa_preb VARCHAR(100) NOT NULL,
  id_mjesto_preb INTEGER NOT NULL CONSTRAINT mjesto_preb_fk REFERENCES mjesto(id_mjesto),
  adresa_bor VARCHAR(100),
  id_mjesto_bor INTEGER CONSTRAINT mjesto_bor_fk REFERENCES mjesto(id_mjesto)
);
```

Slika 6.4: CHECK uvjet u tablici uhicene_osobe

U tablici `policajac_skola` provjeravamo je li godina završetka nakon 1900. godine (razumno je pretpostaviti da policajac koji je ranije od toga završio školu danas više nije živ). Dano je vidljivo na slici 6.3. Na kraju, na slici 6.4 vidimo da se slično kao u tablici `policajac` provjerava spol u tablici `uhicene_osobe` (Republika Hrvatska, u trenutku pisanja, prihvaća samo dva roda, a to su muški i ženski).

6.2 Zadane vrijednosti

U tablici nema puno zadanih vrijednosti, ali su smislene. Sve zadane vrijednosti vidljive su na slici 6.5. Datum uhićenja i datum početka u povijesti zaposlenja postavljeni su na današnji datum, osim ako nije drugačije zadano. Također, atribut o ranijem uhićenju automatski se zadaje na `false` te se kasnije okidačem provjerava može li biti `true`. Više o tome u poglavlju o okidačima.

```
CREATE TABLE uhicenja(
  id_uhicenje SERIAL PRIMARY KEY,
  id_policajac VARCHAR(9) NOT NULL CONSTRAINT policajac_fk REFERENCES policajac(id_policajac),
  id_uhic_osoba VARCHAR(11) NOT NULL CONSTRAINT osoba_fk REFERENCES uhicene_osobe(id_uhic_osoba),
  datum_uhicenja DATE NOT NULL DEFAULT current_date,
  id_djelo INTEGER NOT NULL CONSTRAINT djelo_fk REFERENCES ilegalna_djela(id_djelo),
  opis TEXT,
  ranije_uhicenje BOOLEAN NOT NULL DEFAULT false
);

CREATE TABLE povijest_zaposlenja(
  id_policajac VARCHAR(9) NOT NULL CONSTRAINT policajac_fk REFERENCES policajac(id_policajac),
  id_odjel INTEGER NOT NULL CONSTRAINT odjel_fk REFERENCES odjel(id_odjel),
  id_cin INTEGER NOT NULL CONSTRAINT cin_fk REFERENCES cin(id_cin),
  datum_pocetka DATE NOT NULL DEFAULT current_date,
  datum_zavrsetka DATE,
  PRIMARY KEY (id_policajac, id_odjel, id_cin)
);
```

Slika 6.5: CHECK uvjet u tablici uhicene_osobe

Poglavlje 7

Procedure

Procedure u SQL-u su poput funkcija, ali nemaju opciju vraćanja vrijednosti. Korsite se zbog manjkavosti funkcija da se na njihovom kraju napravi **COMMIT** ili **ROLLBACK** tj. funkcije ne podržavaju transakcije, dok ih procedure podržavaju. Prva procedura koju sam napravio je procedura `dodaj_u_povijest`, vidljiva na slici 7.1. Procedura prima ID policajca i datum te u tablicu `povijest_zaposlenja` dodaje informacije o tom policajcu i datum kada je počeo raditi. Ideja koju sam imao iza ove procedure je da služi za "automatsko" dodavanje novih policajaca u tablicu. Naravno, mogao sam ovu proceduru pretvoriti u funkciju koja vraća **TRIGGER** te napraviti okidač nakon svakog unosa u tablicu `policajac`, ali to bi onda stvorilo neželjeno ponašanje baze podataka jer bi se i unosi starih policajaca unijeli s današnjim datumom. Čak i ako to zaobiđemo redosljednom unosa i kreacije okidača, puno bi lakše bilo implementirati automatsko pokretanje na razini nekakve aplikacije s UI-jem koja će se pokrenuti nakon svakog novog unosa **počevši od nekog trenutka**. Upravo je iz tog razloga, razloga jednostavnosti, ova procedura ostavljena kao ovakva.

```
CREATE OR REPLACE PROCEDURE dodaj_u_povijest(  
    pol_id policajac.id_policajac%type,  
    datum DATE  
)  
LANGUAGE PLPGSQL  
AS  
$$  
DECLARE  
    id_o policajac.id_odjel%type;  
    id_c policajac.id_cin%type;  
BEGIN  
    SELECT id_odjel, id_cin  
    INTO id_o, id_c  
    FROM policajac  
    WHERE id_policajac = pol_id;  
  
    INSERT INTO povijest_zaposlenja(id_policajac, id_odjel, id_cin, datum_pocetka)  
    VALUES  
    (pol_id, id_o, id_c, datum);  
END; $$;
```

Slika 7.1: Procedura `dodaj_u_povijest`

Druga procedura koju sam napravio je `dodaj_mjesto`. U državi sa 127 gradova i 428 općina, dakle 555 jedinica lokalne samouprave, mogućnost stvaranje nove općine ili grada je sveprisutna. Ljudi iz Retfale svakog trenutka mogu proglasiti da su od danas zasebna općina, a mi se s takvim situacijama moramo unaprijed suočiti. Stoga je procedura `dodaj_mjesto` napravljena preventivno, kako bi se dodavanje nove jedinice lokalne samouprave olakšalo budućim korisnicima i upraviteljima baze podataka. Navedena procedura vidljiva je na slici 7.2.

```
CREATE PROCEDURE dodaj_mjesto(  
    naz mjesto.naziv%type,  
    pos_br mjesto.post_broj%type  
)  
LANGUAGE PLPGSQL  
AS  
$$  
BEGIN  
    INSERT INTO mjesto (naziv, post_broj)  
    VALUES  
        (naz, pos_br);  
END; $$;
```

Slika 7.2: Procedura `dodaj_mjesto`

Poglavlje 8

Okidači

Iako je uvjet seminara barem dva okidača, ova baza podataka ima ih dvostruko više zato što je njezin kreator htio što više automatizirati određene procese u bazi podataka. Prvi okidač pri svakom pozivanju `UPDATE` na tablici `policajac` ažurira i unosi podatke u tablicu `povijest_zaposlenja`. Ovaj okidač automatizira promjene zaposlenja i pamti ih u povijesti zaposlenja. Definicija okidača vidljiva je na slici 8.1.

```
CREATE OR REPLACE FUNCTION povijest_zaposlenja_chk()
RETURNS TRIGGER
LANGUAGE PLPGSQL
AS
$$
BEGIN
    UPDATE povijest_zaposlenja
    SET old.datum_zavrsetka = current_date
    WHERE id_policajac = old.id_policajac AND
    id_odjel = old.id_odjel AND
    id_cin = old.id_cin;

    INSERT INTO povijest_zaposlenja (id_policajac, id_odjel, id_cin)
    VALUES
    (new.id_policajac, new.id_odjel, new.id_cin);

    RETURN new;
END;
$$;

CREATE TRIGGER povijest_zaposlenja_trigger
AFTER UPDATE
ON policajac
FOR EACH ROW
EXECUTE FUNCTION povijest_zaposlenja_chk();
```

Slika 8.1: Okidač koji pri svakom ažuriranju u tablici `policajac` ažurira i unosi podatke u tablicu `povijest_zaposlenja`

Drugi okidač nije manje bitan od prvog, on postavlja adresu i mjesto boravišta na ekvivalente prebivališta, ako oni nisu definirani ranije unosom. Upravo zbog ovog okidača, iako su adresa i mjesto boravišta u tablici `uhicene_osobe` opcionalni, oni

postaju opcionalni samo u teoriji, u praksi su uvijek popunjeni nekim podacima. Okidač je vidljiv na slici 8.2.

```
CREATE OR REPLACE FUNCTION preb_bor()
RETURNS TRIGGER
LANGUAGE PLPGSQL
AS
$$
BEGIN
    IF (new.adresa_bor IS NULL AND new.id_mjesto_bor IS NULL) THEN
        UPDATE uhicene_osobe
        SET adresa_bor = adresa_preb
        WHERE id_uhic_osoba = id_uhic_osoba;

        UPDATE uhicene_osobe
        SET id_mjesto_bor = id_mjesto_preb
        WHERE id_uhic_osoba = id_uhic_osoba;

    END IF;
    RETURN new;
END;
$$;

CREATE TRIGGER preb_bor_trigger
AFTER INSERT
ON uhicene_osobe
FOR EACH ROW
EXECUTE FUNCTION preb_bor();
```

Slika 8.2: Okidač koji postavlja boravište na prebivalište, ako boravište nije definirano.

Ni treći okidač nije manje bitan od prethodna dva i on je ostvarenje jednog od početnih ciljeva koje sam htio postići u originalnoj bazi podataka. Ovaj okidač provjerava je li osoba ranije uhićena. Ovaj okidač mi je osobno i najdraži zato što je ostvarenje onoga što sam htio originalno postići. Okidač je vidljiv na slici 8.3.

Četvrti i posljednji okidač radi tako da pri postavljanju voditelja svakog odjela provjerava je li dani radnik zaposlen na tom odjelu; ako nije onda javlja iznimku i vraća *error*. Okidač se može vidjeti na slici 8.4

```

CREATE FUNCTION ranije_uhicen()
RETURNS TRIGGER
LANGUAGE PLPGSQL
AS
$$
BEGIN
    IF (new.id_uhic_osoba IN (SELECT id_uhic_osoba FROM uhicenja)) THEN
        new.ranije_uhicenje = true;
    END IF;
    RETURN new;
END;
$$;

CREATE TRIGGER ranije_uhicen_trigger
BEFORE INSERT
ON uhicenja
FOR EACH ROW
EXECUTE FUNCTION ranije_uhicen();

```

Slika 8.3: Okidač koji provjerava je li osoba ranije uhićena

```

CREATE FUNCTION odjel_voditelj()
RETURNS TRIGGER
LANGUAGE PLPGSQL
AS
$$
DECLARE
    id_o odjel.id_odjel%type;
BEGIN
    SELECT id_odjel
    INTO id_o
    FROM policajac
    WHERE id_policajac = new.id_voditelj;

    IF (id_o != new.id_odjel) THEN
        raise exception 'Policajac ne pripada danom odjelu!';
    END IF;
    RETURN new;
END;
$$;

CREATE TRIGGER voditelj_odjel_trigger
BEFORE UPDATE
ON odjel
FOR EACH ROW
EXECUTE FUNCTION odjel_voditelj();

```

Slika 8.4: Okidač koji provjerava je li osoba ranije uhićena

Poglavlje 9

Indeksi

Iako su indeksi u bazama podataka odlična stvar zato što ubrzavaju pretraživanje, u ovoj bazi nema prevelike potrebe za njima zato što je velika većina primarnih ključeva, kao i onih jedinstvenih ključeva smisleno definirana. Ipak, odlučio sam se uvesti indkse na tablice `mjesto`, `cin` i `ilegalna_djela`. Razlog tome je što ćemo češće u tim tablicama tražiti prema stupcima koji nisu primarni ključevi, stoga posljedično ubrzamo pretraživanje po njima. Ostavio sam sve indkse postavljeni na zadanu vrstu tj. na *B-tree* indkse. Razlog tome je *hash* indksi ovdje neće raditi efikasno kao *B-tree* indksi. Kreiranje indeksa vidljivo je na slici 9.1.

```
CREATE INDEX idx_mjesto_post_broj
ON mesto (naziv, post_broj);

CREATE INDEX idx_cin
ON cin (naziv);

CREATE INDEX idx_djela
ON ilegalna_djela (naziv);
```

Slika 9.1: Definirani indksi

Poglavlje 10

Komentari

Sve su tablice zakomentirane. Komentari na sve tablice mogu se vidjeti na slici 10.1.

```
COMMENT ON TABLE odjel IS 'Tablica svih odjela i svi bitni podatci';
COMMENT ON TABLE završena_skola IS 'Tablica policijskih škola';
COMMENT ON TABLE vozilo IS 'Tablica svih vozila u policijskoj stanici';
COMMENT ON TABLE policajac IS 'Tablica svih podataka o policajcima';
COMMENT ON TABLE policajac_vozilo IS 'Vezna tablica između tablica policajac i vozilo';
COMMENT ON TABLE policajac_skola IS 'Vezna tablica između tablica policajac i završena_skola';
COMMENT ON TABLE povijest_zaposlenja IS 'Tablica koja prati povijest zaposlenja svakog policajca';
COMMENT ON TABLE ilegalna_djela IS 'Tablica počinjenih ilegalnih djela';
COMMENT ON TABLE uhićenja IS 'Tablica u kojoj su podatci o svakom uhićenju';
COMMENT ON TABLE uhićene_osobe IS 'Tablica podataka o uhićenim osobama';
COMMENT ON TABLE mjesto IS 'Tablica mjesta';
```

Slika 10.1: Komentari na tablice

Poglavlje 11

Zaključak

Iako ova baza podataka postoji u puno opsežnijem obliku, vjerujem da je ovaj projekt dovoljno vjeran stvarnom modelu baze podataka kako bi prošao kao dovoljno opširan. Bilo je ovdje izazovnih stvari, poput modeliranja dvostruke veze između tablica `odjel` i `policajac`, kao i svega vezanog uz tablicu `povijest_zaposlenja`. Sviđa mi se što je ovim projektom pokriveno sve što smo radili iz SQL-a na kolegiju, poput indeksa i komenatara na tablice. Najteže mi je bilo smisliti 20 upita za ovu bazu podataka, teško mi je bilo smisliti 5 koliko-toliko smislenih upita za ovu bazu podataka. Svakako sam iz seminara puno naučio, što u SQL-u, što o radu u L^AT_EX, a drago mi je da sam ostavio dovoljno mjesta na eventualnu nadogradnju na program sa cijelim UI i proširenjem rada u npr. Pythonu. Vjerujem da ovaj projekt ima opciju proširenja na smisleniji program. Također, naučio sam puno o Hrvatskoj pošti i poštanskim brojevima u Republici Hrvatskoj.